

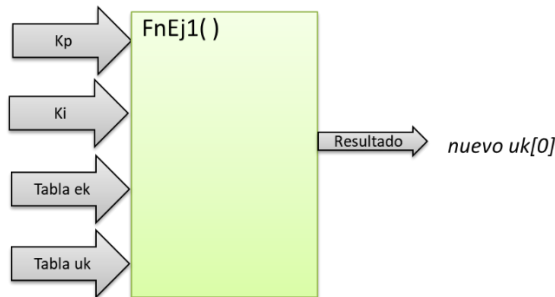
Master en Ingeniería Mecatrónica

Computadores y Programación

Examen Ordinario – Enero 2025

- 1) Escribir una función que, dadas dos valores K_p y K_i , y las tablas de datos actual y anteriores de error y acción de control, calcule el nuevo valor de la acción de control para un regulador PI definido como:

$$R(z) = \frac{(K_i + K_p) - K_i \cdot z^{-1}}{1 - z^{-1}}$$



Recordatorio. Para un $R(z)$ genérico definido como:

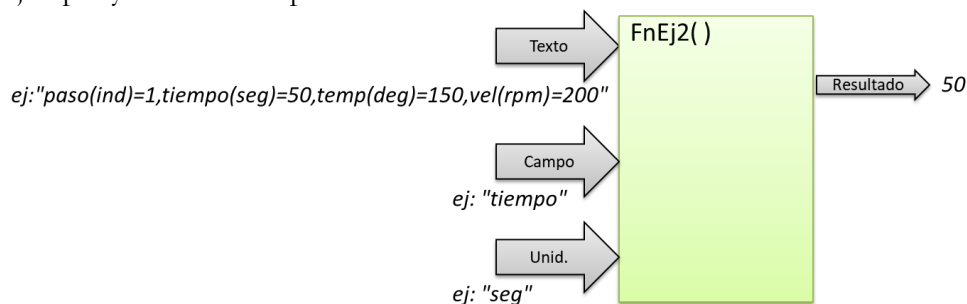
$$R(z) = \frac{U(z)}{E(z)} = \frac{b_0 + b_1 \cdot z^{-1} + \dots + b_m \cdot z^{-m}}{1 + a_1 \cdot z^{-1} + \dots + a_n \cdot z^{-n}}$$

la nueva acción de control se calcula aplicando la ecuación en diferencias:

$$u_k = b_0 \cdot e_k + b_1 \cdot e_{k-1} + \dots + b_m \cdot e_{k-m} - (a_1 \cdot u_{k-1} + \dots + a_n \cdot u_{k-n})$$

- 2) Realizar una función que, dada una cadena de caracteres con varios campos, donde el formato de cada campo siempre es: **nombre(unidades)=número real** extraiga y obtenga de la misma el número real correspondiente al nombre y unidades pasados como argumentos, o el valor **-1** en caso de no encontrarlo.

Ejemplo y resultados esperados:



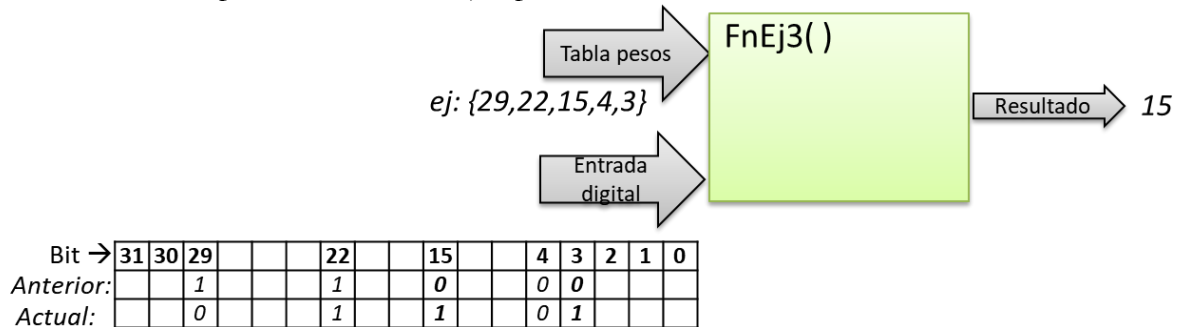
Master en Ingeniería Mecatrónica

Computadores y Programación

Examen Ordinario – Enero 2025

- 3) Realizar una función que, dados el estado anterior y actual de los bits de entrada digital de un computador, y una tabla de enteros con los pesos de los bits a chequear, calcule y devuelva:
1.5 puntos El peso del 1^{er} bit de los indicados en la tabla que ha cambiado de 0 a 1 en la entrada digital, si hay alguno
-1, si ningún bit de los indicados ha cambiado de 0 a 1 en la entrada digital

Resultado para los valores de ejemplo:



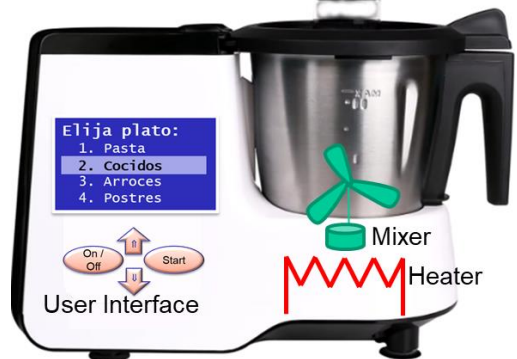
Master en Ingeniería Mecatrónica

Computadores y Programación

Examen Ordinario – Enero 2025

4)
5 puntos

Se desea automatizar el funcionamiento de un robot de cocina, que dispone de los siguientes elementos:

 El diagrama muestra un robot de cocina con una pantalla LCD que dice 'Elija plato: 1. Pasta, 2. Cocidos, 3. Arroz, 4. Postres'. Debajo de la pantalla hay tres botones: 'On/Off', 'Start' y un botón con flechas hacia arriba y hacia abajo. A la izquierda de estos botones está el texto 'User Interface'. A la derecha del robot hay un 'Mixer' (representado por un icono de paleta) y un 'Heater' (representado por un icono de resistencia eléctrica).	▣ Accionamientos: Un motor para batir a diferentes velocidades Una resistencia para calentar ▣ Sensores: Velocidad del motor Temperatura de la mezcla ▣ Interfaz usuario: Pulsadores On/Off, Start, Up y Down Led On/Off
---	--

Para el funcionamiento, el usuario selecciona mediante los botones del UI el plato a cocinar, y una vez seleccionado se dispondrá de líneas de texto que indican los pasos para cocinar dicho plato. Cada paso se codifica en una línea, con un formato como el siguiente:

```
paso(ind)=1, tiempo(seg)=50, temp(deg)=150, vel(rpm)=200
paso(ind)=2, tiempo(seg)=300, temp(deg)=100, vel(rpm)=100
paso(ind)=3, tiempo(seg)=500, temp(deg)=100, vel(rpm)=500
...
```

El programa principal `main()` se encarga de:

- ▣ Inicializar variable `_estado` a **PARADO**.
- ▣ Disponer de variables para valor actual y anterior de pulsadores, inicializar a 0.
- ▣ Realizar un bucle con los siguientes contenidos, en función del estado actual:
 - Siempre: actualizar valores de entrada digital actual y anterior. Aplicar función ejercicio 3 para determinar qué botón se ha pulsado.
 - En función del estado:
 - **PARADO**: Led On/Off apagado, tensión motor U_m a cero, resistencia de calentamiento desactivada, detener interrupción temporizada.
Si se ha pulsado ON/OFF, pasar a estado **ESPERA_SELECCION**, con valor `ind_seleccion=1` y Led On/Off arrancado.
 - **ESPERA_SELECCION**. Según el botón pulsado:
 - ▣ Botón [↓] → incrementar `ind_seleccion`
 - ▣ Botón [↑] → decrementar `ind_seleccion`
 - ▣ Botón [Start] → Pasar a estado **COCINANDO**, `_ind_paso=1`
 - ▣ Botón [On/Off] → regresar a estado **PARADO**
 - **COCINANDO**:
 - ▣ Si se ha pulsado botón [On/Off] → regresar a estado **PARADO**.
 - ▣ Si no:
 - Llamar a la función `GetLinea()` (ver apartado anexos) para obtener el contenido en forma de texto del paso a procesar.
 - Si `GetLinea()` devuelve OK, procesar con función ejercicio 2 para obtener consignas de temperatura, velocidad, y tiempo máximo, e iniciar contador de tiempo. Lanzar interrupción temporizada cada 1seg.
 - Si, por el contrario, `GetLinea()` devuelve FAIL, volver al estado **PARADO**.

Master en Ingeniería Mecatrónica

Computadores y Programación

Examen Ordinario – Enero 2025

En la interrupción temporizada se ejecutan sendos controladores para temperatura y velocidad, de acuerdo con las consignas establecidas en cada momento por la secuencia.

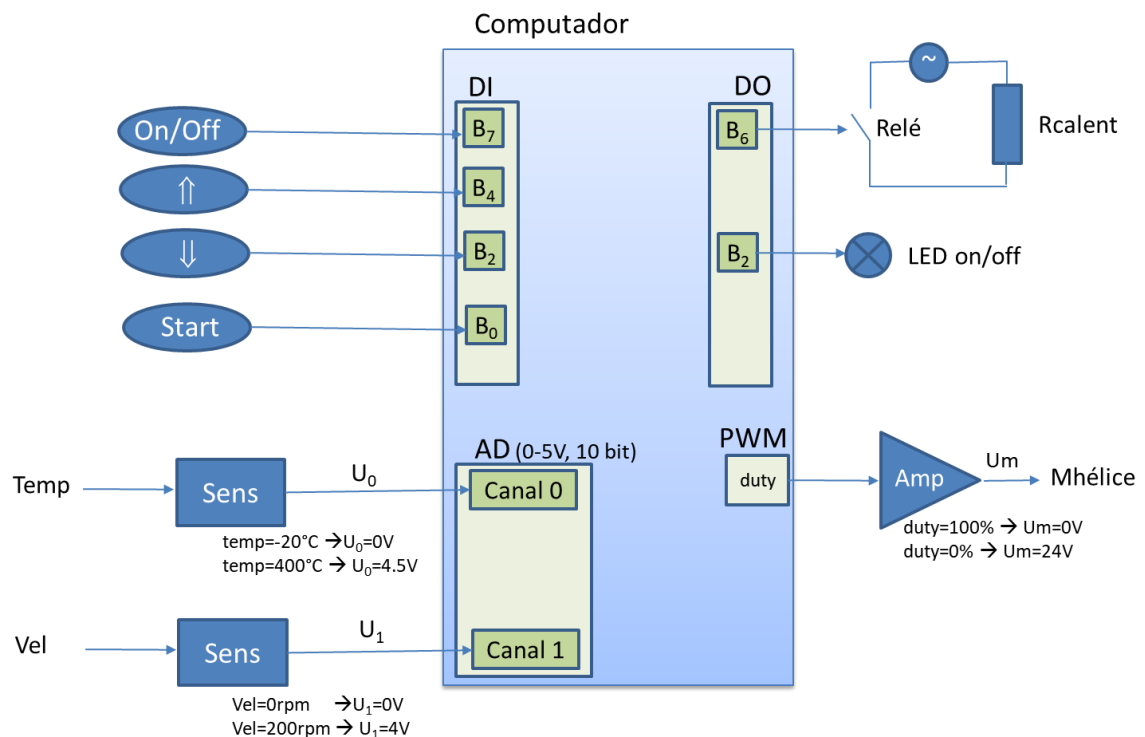
- El control de temperatura se realiza mediante un regulador Todo/Nada, que activa/desactiva la resistencia de calentamiento en función de la temperatura consigna y de la temperatura medida.
- El control de velocidad de la hélice se realiza mediante un regulador PI con parámetros K_p, K_i definidos como constantes, mediante la función del ejercicio 1:

$$R(z) = \frac{(K_i + K_p) - K_i z^{-1}}{1 - z^{-1}} \rightarrow uk[0] (V) = F_{nEj1}(K_i, K_p, ek(rpm), uk (V));$$

Si $uk[0]$ y $uk[1]$ son ambos superiores a 50V, se supone que hay un atasco. En este caso, poner `_ind_paso=1000` para que el programa principal termine la ejecución.

- Se incrementa contador de tiempo. Si el tiempo excede el máximo, incrementar `_ind_paso` en 1.

Diagrama de conexiones:



Master en Ingeniería Mecatrónica

Computadores y Programación

Examen Ordinario – Enero 2025

Librería auxiliar para E/S específica del computador destino:

```
int LeerCanalAD(int n_canal);    // Obtiene valor de canal A/D, 10 bits 0/5V
void InitTemporizador(int tm_ms,void (*FnISR)()); // Arranca temporizador
void FinTemporizador(); // Detiene temporizador
int LeerDI(); // Lee estado de entradas digitales
void EscribirDO(int do); // Actualiza estado de salidas digitales
void ActivarPWM(float duty_0_a_1); // Escribe PWM con el duty deseado
int GetLinea(int ind_selec,int ind_paso,char* dest); // Rellena en dest el contenido de
la línea de texto con las consignas y tiempo máximo para una selección (plato) y un índice
de paso de ese plato. Si se encuentra, devuelve 1 (OK). Si no se encuentra, devuelve 0
(FAIL). El programador debe disponer de espacio para 80 caracteres en el argumento dest.
```

Algunas funciones de C:

```
int atoi(const char* cad);          // Devuelve entero equivalente a cadena
double atof(const char* cad);       // Devuelve real equivalente a cadena
double strtod(const char* cad,char** next); // Id. a atof() y guarda en next puntero
// a final de conversión
int strlen(const char* cadena);      // Devuelve longitud de cadena
char* strcpy(char* dst,const char* src); // Copia cadena fuente en destino
char* strncpy(char* dst,const char* src,int n); // Id. Máximo n caracteres
char* strcat(char* dst,const char* src); // Concatena cadena Fuente a destino
char* strncat(char* dst,const char* src,int n); // Id. Máximo n caracteres
char* strchr(const char* cad,char c); // Busca caracter en cadena, devuelve puntero
// a la primera ocurrencia o NULL si no está
char* strstr(const char* cad,const char* busca); // Id. buscando cadena
int strcmp(const char* c1,const char* c2); // Compara cadenas, devuelve 0 si iguales
int strncmp(const char* c1,const char* c2,int n); // Compara cadenas hasta max n
// caract, devuelve 0 si iguales
char* gets(char* destino);          // Lee cadena de consola, almacena en destino
void* malloc(int n_bytes);          // Asigna memoria para n bytes
void free(void* ptr);               // Libera memoria asignada
```

Master en Ingeniería Mecatrónica

Computadores y Programación

Examen Ordinario – Enero 2025

APELLIDOS Y NOMBRE: _____

Cuestiones (responder aquí y entregar esta hoja con nombre y apellido):

(0.25 ptos Acertada, -0.15 ptos Fallada, 0 ptos No Contestada)

- a) ¿Cuál de estos formatos es correcto para una función que, dada una tabla de datos, debe devolver un número real y un código de error ?

☐ float FnA(const float t[],int n,int *err);	☐ float,int FnA(float t[] const,int const n);
☐ float FnA(const float t[],int n,int err);	☐ {float,int} FnA(const float t[],int n);

- b) ¿Qué ocurre con el código siguiente para convertir los valores de una tabla en sus cuadrados ?

<pre>void Cuadrados(const int t[],int n) { int i; for (i=0;i<n;i++) t[i] *= t[i]; } ... int datos[4]={0,2,4,6}; Cuadrados(datos[],4);</pre>	☐ No se puede compilar, la declaración del puntero t no puede ser const ☐ No se puede compilar, escribir datos[] en la llamada a Cuadrados() es inválido ☐ Las dos anteriores son ciertas ☐ Ninguna de las anteriores es cierta
--	--

- c) ¿Cuánto vale la variable j tras ejecutar el código siguiente ?

<pre>#define N 8 int j; for (j=0 ; j<N ; j++) { } ¿Cuánto vale j aquí?</pre>	☐ j vale 8 ☐ j vale 7 ☐ No se puede usar j fuera del bucle ☐ No se puede compilar, el bucle for está mal construido
---	---

- d) ¿Qué hace el código siguiente?

<pre>int main() { char txt[40]; printf("Escriba texto: "); gets(txt); if (txt>="HOLA") printf("Saludo\n"); return 0; }</pre>	☐ Escribe en pantalla Saludo si el usuario introduce por teclado un texto que comience por la palabra HOLA ☐ Escribe en pantalla Saludo si el usuario introduce por teclado cualquier texto que contenga la palabra HOLA ☐ Escribe en pantalla Saludo si el usuario introduce por teclado cualquier texto que no contenga la palabra HOLA ☐ Ninguna de las anteriores
---	--